educación IT
CODE YOUR FUTURE

Preparación del entorno

¿Cómo comenzar?

La preparación del entorno de programación es el primer paso para crear *software* de manera eficiente. Comienza con la **instalación del lenguaje de programación** elegido. Para ello, siempre es recomendable **descargar la versión más reciente** desde el sitio web oficial.

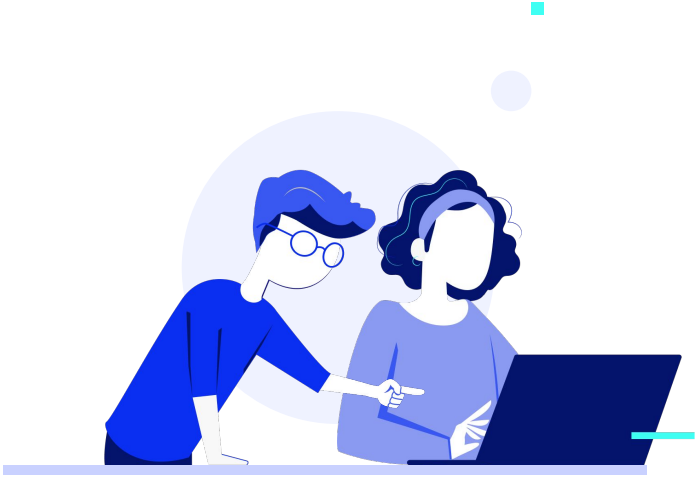
Además, hay que prestar especial atención a las **configuraciones del sistema operativo** que permitan ejecutar los comandos desde cualquier ubicación.

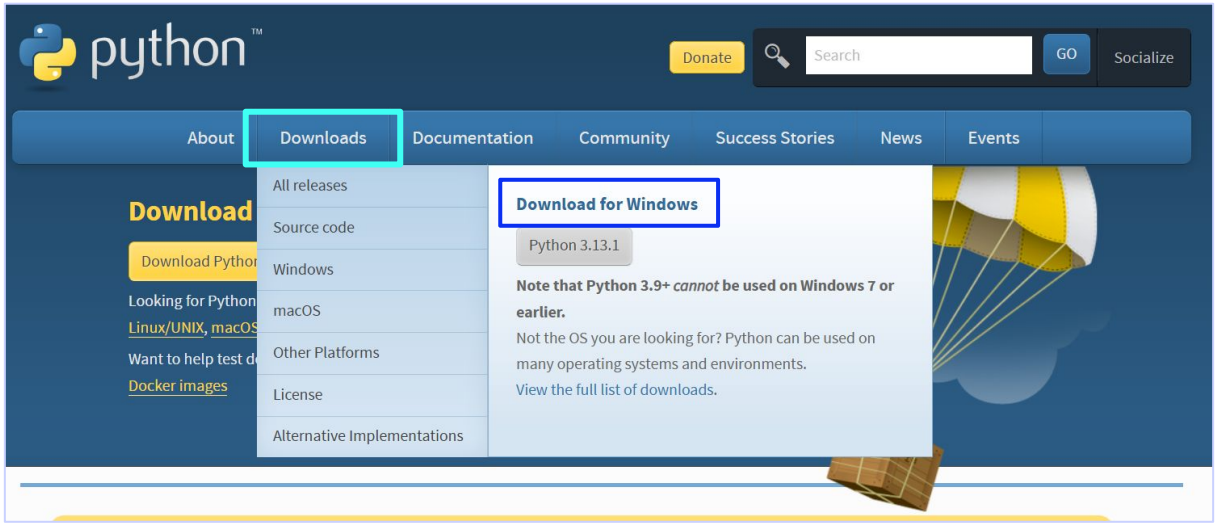


Instalación de Python

Antes de empezar a programar, se debe instalar las aplicaciones necesarias para esta tarea.

Lo principal para este curso es tener instalado **Python**. Para ello, dirígete a la página oficial del lenguaje (python.org) y sigue los pasos que se muestran a continuación.





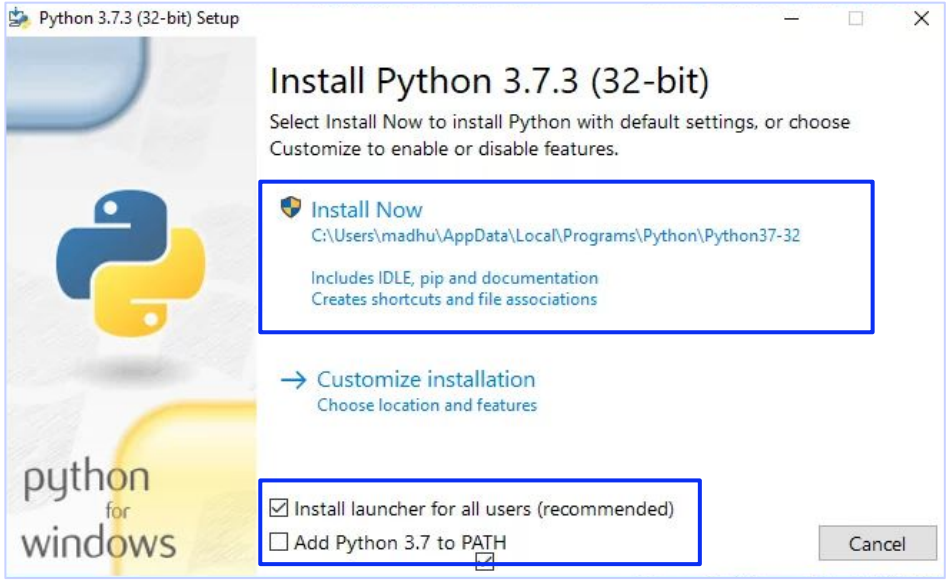
> Al momento en que escribimos este material, la última versión estable de Python es la 3.13.1. Como el lenguaje se actualiza con bastante frecuencia, es probable que exista una versión más reciente. Puedes descargarla sin problemas.

Una vez descargado el instalador, se procede a ejecutarlo.

Debes asegurarte de **ejecutarlo como administrador** y también seleccionar:

1. Install laucher for all users
2. Add Python 3.x to PATH

Luego se realiza una instalación normal, siguiendo los pasos hasta el final.



Lo óptimo es **instalar un entorno virtual para cada proyecto**, de modo que puedas crear ambientes de trabajo aislados con dependencias específicas.

La instalación de un **gestor de paquetes** como pip y la **configuración de un IDE** (luego veremos en qué consiste) completarán el entorno de desarrollo.

Es importante **verificar la instalación** ejecutando comandos básicos como `python --version` y `pip --version` en la terminal, lo que confirmará que todo está correctamente configurado y listo para iniciar la programación.

Pip es un **gestor de paquetes de Python** que permite instalar, actualizar y administrar bibliotecas y dependencias externas de manera sencilla y automática mediante comandos de línea.



Editores de código

Se puede escribir código en cualquier lugar; sin embargo, hay algunos programas pensados específicamente para esta tarea que son de gran ayuda al momento de programar. Se conocen como **editores de código**.

No debes confundirlos con los **entornos de desarrollo integrado**: aplicaciones más completas que, además de un editor de código, incluyen otras herramientas como sistemas para trabajar en proyectos grupales con otros programadores.



Visual Studio Code

Existen varios editores de código con los que trabajar:

- Geany
- PyCharm
- Visual Studio
- VS Code
- Sublime Test

Hoy en día, el editor de código más usado es **Visual Studio Code** para programar en varios lenguajes, y es el que utilizaremos en este curso. VS Code es un potente editor de código, con *plugins* y herramientas muy interesantes.



Si te interesa, descárgalo desde la página oficial Visual Studio Code. Puedes pedirle al profesor que te ayude a configurarlo.

Entornos de desarrollo integrado (IDE)

Los entornos de desarrollo integrado (IDE, por sus siglas en inglés) son herramientas para programadores que **simplifican y optimizan el proceso de escritura, depuración y ejecución de código**.

Un IDE **combina múltiples funcionalidades en una única plataforma**, ofreciendo un espacio de trabajo completo y eficiente para el desarrollo de *software*.

Las principales características de un IDE incluyen:

- **Editor de código** con resaltado de sintaxis, que permite una lectura más clara y rápida del código.
- **Autocompletado inteligente** que sugiere métodos y variables.
- **Depurador integrado** para identificar errores.
- **Herramientas de gestión de proyectos** que facilitan la organización del trabajo.

Intérpretes y compiladores

Es importante saber diferenciar un IDE, es decir, una herramienta de *software* que proporciona un espacio de trabajo integral para programadores, de un **intérprete** y un **compilador**.

Estas son herramientas más específicas que se encargan de **transformar el código escrito** por un programador **en instrucciones que una computadora puede ejecutar**.

Veamos el cuadro de la siguiente *slide.*



Intérprete	Compilador
Lee y ejecuta el código línea por línea en tiempo real, traduciendo cada instrucción de manera inmediata, lo que permite una ejecución más flexible pero potencialmente más lenta.	Traduce todo el código fuente a lenguaje máquina de una vez, generando un archivo ejecutable independiente, lo que suele resultar en una ejecución más rápida, pero requiere un paso adicional de compilación antes de ejecutar el programa.

> En Python, por ejemplo, se utiliza principalmente un intérprete, aunque internamente existe un proceso de compilación a *bytecode* para optimizar la ejecución.